

VIA University College

Kløverly – Projektbeskrivelse

Mehmet Sahin (360972) og Chadi El-Khatib (361354)

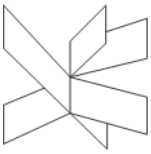
Vejledere: Michael Viuff & Troels Mortensen

Antal tegn: 8.152 inkl. mellemrum

Softwareingeniør Trainee

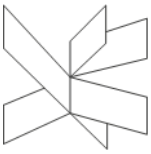
1. semester (SEP1) – reeksamen

16-10-2026



Indhold

1. Problemfelt 1
2. Problemformulering 2
3. Afgrænsning 3
4. Valg af metoder 4
Kravspecifikation 4
Analyse 4
Design 4
Implementering 4
Test 5
Samarbejde og brug af AI 5
5. Tidsplan 6
6. Risikovurdering 7
7. Referencer 8



1. Problemfelt

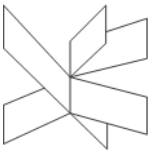
Kløverly er en økologisk landsby med 42 beboere, som dyrker egne grøntsager og deler ting og tjenester med hinanden (VIA University College, 2025a). Meget af det, beboerne gør for fællesskabet og for klimaet, er "usynligt arbejde": at cykle i stedet for at køre, at lave mad til fællesspisning eller at hjælpe en nabo. Den slags handlinger giver sjældent et synligt resultat med det samme, og det kan gøre det svært at holde motivationen oppe (Brick et al., 2021).

Koordinatoren, Grønne Bob, holder i dag styr på landsbyens aktiviteter i en notesbog. Han beskriver tre slags opgaver, som skal behandles forskelligt:

- **Grønne tiltag** giver point til en fælles pulje, landsbyens "klimakapital", og skal vises uden navne som inspiration for andre.
- **Fællesopgaver** giver personlige point til den beboer, der udfører opgaven.
- **Bytteopgaver** er ydelser, som beboerne bytter, og de betales med personlige point.

Bob vil gerne have et system, der kan holde styr på opgaverne og de to slags point, og en hjemmeside, hvor alle kan læse om Kløverly og blive inspireret. Samtidig lægger han vægt på, at fællesskabet foregår "uden konkurrence eller tvang". Et pointsystem kan let komme til at virke som en rangliste, så det er en del af problemet at finde ud af, hvad der må vises, og for hvem. Bob og eventuelt Don er de eneste, der skal bruge programmet, og det skal kun ligge på Bobs computer.

Bob har også idéer, som han ikke selv har gjort færdige: en stor skærm i landsbyen, mål for fællespuljen, nulstilling af point med faste intervaller og et pointboost til beboere, der sjældent deltager. De idéer tager vi stilling til i afgrænsningen.

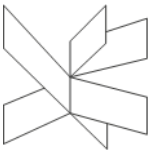


2. Problemformulering

Hvordan kan vi udvikle et IT-system, der hjælper koordinatoren i Kløverly med at registrere landsbyens tre typer opgaver og de to typer point, og som samtidig formidler de grønne tiltag til omverdenen uden at udstille den enkelte beboer?

Underspørgsmål:

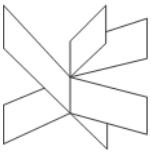
- Hvilke krav kan udledes af interviewet med Grønne Bob, og hvordan kan de beskrives, så de kan testes?
- Hvordan kan pointreglerne for de tre opgavetyper modelleres, så personlige point aldrig bliver negative, og historikken bevares?
- Hvordan kan data fra programmet vises på en hjemmeside ved hjælp af JavaScript uden at vise personoplysninger?
- Hvordan kan vi dokumentere, at løsningen opfylder kravene?



3. Afgrænsning

Projektet gennemføres på én uge. Vi prioriterer derfor det, Bob beskriver som nødvendigt: registrering af beboere, de tre opgavetyper, de to slags point og hjemmesiden. Følgende er bevidst udeladt:

- **Login og adgang for beboerne.** Kun Bob og Don bruger programmet, og det ligger kun på Bobs computer.
- **Den store skærm i landsbyen.** Det er ikke afklaret, hvad skærmen skal vise, og uden det kan vi ikke sikre, at den ikke skaber konkurrence.
- **Mål for fællespuljen og pointboost.** Bob har ikke selv besluttet reglerne. Et ekstra boost kan gives manuelt som en pointkorrektion.
- **Automatisk nulstilling med faste intervaller.** Koordinatoren kan nulstille en beboers point manuelt.
- **Indrapportering fra beboerne selv.** Hjemmesiden viser kun information.



4. Valg af metoder

Vi følger vandfaldsmodellen, hvor krav og analyse, design, implementering og test kommer efter hinanden (Royce, 1970). Modellen er en del af kursets læringsmål (VIA University College, 2025b), og den passer til et kort projekt, hvor kravene kan lægges fast tidligt. Mellem faserne har vi beslutningspunkter, Mål A–D, hvor vi tjekker, at det, vi har lavet, hænger sammen, før vi går videre. Hvis vi finder fejl i en tidligere fase, går vi tilbage og retter den i stedet for at lade den stå.

Kravspecifikation

Kravene udledes af interviewet med Bob i casen. Vi gennemgår interviewet sætning for sætning og skriver hvert ønske om til et funktionelt krav på formen "Systemet skal kunne …" med citatet ved siden af. De ikke-funktionelle krav skrives, så hvert krav kan måles i en test. Alle krav får et ID (FR og NFR), så de kan følges gennem resten af projektet.

Analyse

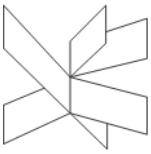
Kravene omsættes til use cases med ID (UC), et use case-diagram og use case-beskrivelser med hovedscenarie og alternativer. Vi laver aktivitetsdiagrammer for forløbene og en domænemodel med de begreber, Bob bruger. Udbyttet er en fælles forståelse af, hvad systemet skal kunne, før vi skriver kode.

Design

Domænemodellen udvikles til et klassediagram, og de vigtigste forløb vises i sekvensdiagrammer. Vi deler programmet op efter Model-View-Controller, så reglerne for point ligger i modelklasserne og kan testes uden brugerfladen. Hjemmesiden får sine data fra en separat datafil, som kun indeholder anonyme oplysninger.

Implementering

Programmet skrives i Java 21 med JavaFX til brugerfladen og Maven til at bygge projektet. Hjemmesiden laves med HTML, CSS og JavaScript. Vi bruger Git og GitHub fra første dag, så vi begge arbejder i den samme kodebase og committer løbende med



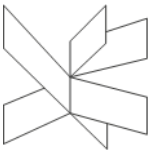
beskeder, der henviser til krav-ID'erne. Modelklasserne og deres regler laves først, så de kan testes, mens resten bliver bygget.

Test

Vi tester på to niveauer. Pointreglerne testes med JUnit 5, og hele programmet testes som black box ud fra use case-beskrivelserne. Hjemmesiden testes for sig, især for at den ikke viser navne. En sporbarhedsmatrix forbinder hvert krav med en use case, en klasse og en test.

Samarbejde og brug af AI

Vi skriver en gruppekontrakt den første dag. Vi har kontakt hver dag og aftaler, hvilke opgaver der skal være færdige. Vi må gerne bruge generativ AI som sparring, men alt, hvad AI laver, skal gennemgås og testes af os, og vi skal begge kunne forklare det. Brugen beskrives i procesrapporten.

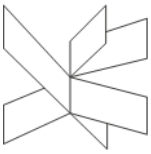


5. Tidsplan

Projektet løber fra mandag den 12. oktober til aflevering fredag den 16. oktober 2026 kl. 13.00. Den mundtlige eksamen er den 22. oktober. Tidsplanen er derfor en plan dag for dag med et mål for hver dag:

Dag	Fase	Aktiviteter	Mål
Mandag 12/10	Opstart, krav og analyse	Gruppekontrakt, brainstorm og aktører, kravspecifikation, use cases, domænemodel. Maven-projekt, GitHub og de første modelklasser.	Mål A: Krav og domænemodel på plads
Tirsdag 13/10	Design og implementering	Klassediagram og sekvensdiagrammer. JavaFX-brugerflade og gem og indlæs.	Mål B: Beboere og opgaver kan registreres
Onsdag 14/10	Implementering	Pointregler for alle tre opgavetyper, hjemmeside med JavaScript, aktivitetsdiagrammer.	Mål C: Alle krav er implementeret
Torsdag 15/10	Test og dokumentation	Unittests, black box-tests og test af hjemmesiden. Sporbarhedsmatrix, projektrapport og procesrapport.	Mål D: Alle tests er gennemført
Fredag 16/10	Aflevering	Sidste gennemlæsning, videopræsentation og aflevering kl. 13.00.	Projektet er afleveret

Tabel 1: Tidsplan for projektugen.

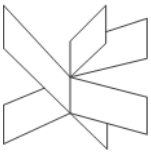


6. Risikovurdering

Tabellen viser de risici, vi ser for projektet. Risikotallet er konsekvens gange sandsynlighed på en skala fra 1 til 5.

Hvad kan gå galt?	K	S	Risiko	Forebyggende handling	Ansvarlig
Tiden slår ikke til, fordi projektet kun varer fem dage.	5	4	20	Plan dag for dag med et mål for hver dag. Afgræns tidligt, og stop nye funktioner onsdag.	Begge
Én af os kender kun koden og den anden kun rapporten, så vi ikke begge kan forklare projektet.	5	3	15	Løbende gennemgang af hinandens arbejde og en fælles gennemgang til sidst.	Begge
Diagrammer, kode og rapport passer ikke sammen.	4	3	12	Krav-ID'er overalt og en sporbarhedsmatrix. Diagrammerne tjekkes mod koden.	Chadi
Materiale fra AI er forkert eller ikke forstået.	4	3	12	Alt AI-materiale gennemgås og testes, før det bruges.	Chadi
Vi arbejder på forskellige tidspunkter ved siden af job og anden undervisning.	3	4	12	Daglig kontakt og en fast aftale om dagens opgaver.	Mehmet
Kravene er uklare, fordi Bob ikke selv har besluttet alle regler.	3	3	9	Kun det, Bob har bekræftet, bliver til krav. Resten afgrænses.	Mehmet
Arbejde går tabt ved sammenfletning i Git.	4	2	8	Små commits, hent altid de nyeste ændringer først, og aftal hvem der arbejder i hvilke filer.	Mehmet
Sygdom eller fravær.	4	1	4	Begge kender hele projektet, og alt ligger på GitHub.	Begge

Tabel 2: Risikovurdering. K = konsekvens, S = sandsynlighed.



7. Referencer

Brick, C., Bosshard, A., & Whitmarsh, L. (2021). Motivation and climate change: A review. Current Opinion in Psychology, 42, 82–88. https://doi.org/10.1016/j.copsyc.2021.04.001

Royce, W. W. (1970). Managing the development of large software systems. I Proceedings of IEEE WESCON (s. 1–9). IEEE.

VIA University College. (2024). Guide til projektarbejde – Projektbeskrivelse. VIA Ingeniøruddannelser.

VIA University College. (2025a). Kløverly – SW-SEP1-case, efterår 2025.

VIA University College. (2025b). Kursusbeskrivelse for TS-SEP1.